

Building TF-IDF (Term Frequency-Inverse Document Frequency) involves preprocessing text, calculating how often a word appears in a document (TF), determining its rarity across all documents (IDF), and then multiplying these scores to find a word's significance, effectively turning text into numerical vectors for machine learning, often using Python libraries like Scikit-learn's `TfidfVectorizer` for ease. [🔗](#)

Steps to Build TF-IDF

1. Text Preprocessing: Clean your text data by:

- Lowercasing everything.
- Removing punctuation and special characters.
- Tokenizing (splitting) text into words.
- Removing common "stop words" (like 'the', 'a', 'is') that don't add much meaning.

2. Calculate Term Frequency (TF): For each document, find the frequency of a word.

- **Formula:** $TF(\text{term}, \text{document}) = (\text{Number of times 'term' appears in document}) / (\text{Total words in document})$.

3. Calculate Inverse Document Frequency (IDF): Measure how unique a word is across the entire corpus.

- **Formula:** $IDF(\text{term}) = \log(\text{Total number of documents} / \text{Number of documents containing 'term'})$.
- *Note:* Add 1 to the denominator to avoid division by zero, or use $\log(N/df+1)$ for smoothing.

4. Calculate TF-IDF Score: Multiply the TF and IDF scores.

- **Formula:** $TF-IDF(\text{term}, \text{document}) = TF(\text{term}, \text{document}) * IDF(\text{term})$.

5. Create the Matrix: Combine these scores into a matrix where rows are documents, columns are unique words (vocabulary), and values are the TF-IDF scores. [🔗](#)

Recipe 3-6. Converting Text to Features Using TF-IDF

The above-mentioned text-to-feature methods have a few drawbacks, hence the introduction of TF-IDF. The following are some of the disadvantages.

- Let's say a particular word appears in all the corpus documents. It achieves higher importance in our previous methods, but that may not be relevant to your case.
- TF-IDF reflects on how important a word is to a document in a collection and hence normalizes words that frequently appear in all the documents.

Problem

You want to convert text to features using TF-IDF.

Solution

Term frequency (TF) is the ratio of the count of a particular word present in a sentence to the total count of words in the same sentence. TF captures the importance of the word irrespective of the length of the document. For example, a word with a frequency of 3 in a sentence with 10 words is different from when the word length of the sentence is 100 words. It should have more importance in the first scenario, which is what TF does.

$$\text{TF}(t) = (\text{Number of times term } t \text{ appears in a document}) / (\text{Total number of terms in the document}).$$

Inverse document frequency (IDF) is a log of the ratio of the total number of rows to the number of rows in a particular document in which a word is present. $IDF = \log(N/n)$, where N is the total number of rows, and n is the number of rows in which the word was present.

IDF measures the rareness of a term. Words like *a* and *the* show up in all the corpus documents, but rare words are not in all documents. So, if a word appears in almost all the documents, that word is of no use since it does not help with classification or information retrieval. IDF nullifies this problem.

TF-IDF is the simple product of TF and IDF that addresses both drawbacks, making predictions and information retrieval relevant.

$$TF\text{-}IDF = TF * IDF$$

How It Works

Follow the steps in this section.

Step 6-1. Read the text data

The following is a familiar phrase.

```
Text = ["The quick brown fox jumped over the lazy dog.",
"The dog.",
"The fox"]
```

Step 6-2. Create the features

Execute the following code on the text data.

```
#Import TfidfVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
#Create the transform
vectorizer = TfidfVectorizer()
#Tokenize and build vocab
vectorizer.fit(Text)
```

```
#Summarize

print(vectorizer.vocabulary_)
print(vectorizer.idf_)
```

This is the result.

```
Text = ["The quick brown fox jumped over the lazy dog.",
"The dog.",
"The fox"]

{'the': 7, 'quick': 6, 'brown': 0, 'fox': 2, 'jumped': 3, 'over': 5,
'lazy': 4, 'dog': 1}

[ 1.69314718  1.28768207  1.28768207  1.69314718  1.69314718
 1.69314718  1.69314718  1.   ]
```

Observe that the appears in all three documents, so it does not add much value. The vector value is 1, which is less than all the other tokens.

All the methods or techniques you have looked at so far are based on frequency. They are called frequency-based embeddings or features. The next recipe looks at prediction-based embeddings, typically called word embeddings.